are that the objects must respond to succ by returning the next object in sequence and the objects must be comparable using <=> (as described in Mixins, on page 107).

In reality, this isn't something you do very often, so examples tend to be a bit contrived. Here's one—a class that presents numbers that are powers of 2. Because it defines <=> and succ, we can use objects of this class in ranges:

```
class PowerOfTwo
  attr_reader :value
  def initialize(value)
    @value = value
  end

  def <=>(other)
    @value <=> other.value
  end

  def succ
    PowerOfTwo.new(@value + @value)
  end

  def to_s
    @value.to_s
  end
end

p1 = PowerOfTwo.new(4)
p2 = PowerOfTwo.new(32)

puts (p1..p2).to_a
```

produces:

```
4
8
16
32
```

Ranges as Intervals

A final use of the versatile range is as an interval test: seeing whether some value falls within the interval represented by the range. We do this using ===, the case equality operator, which is equivalent to the include? methods for boolean testing. Ranges also provide the cover? method, which is identical to include? for numbers, but for non-numeric sequences behaves differently. The cover? method includes any item between the start and end of the range even if the item is not in the range itself.

```
(1..10) === 5             # => true
(1..10) === 15            # => false
(1..10) === 3.14159       # => true
('a'..'j') === 'c'        # => true
('a'..'j') === 'z'        # => false
('a'..'j').include?('c')  # => true
('a'..'j').include?('bb') # => false
('a'..'j').cover?('bb')   # => true
```

Since case statements use triple-equals for comparisons, ranges are often used as a convenient shortcut for branch conditions.